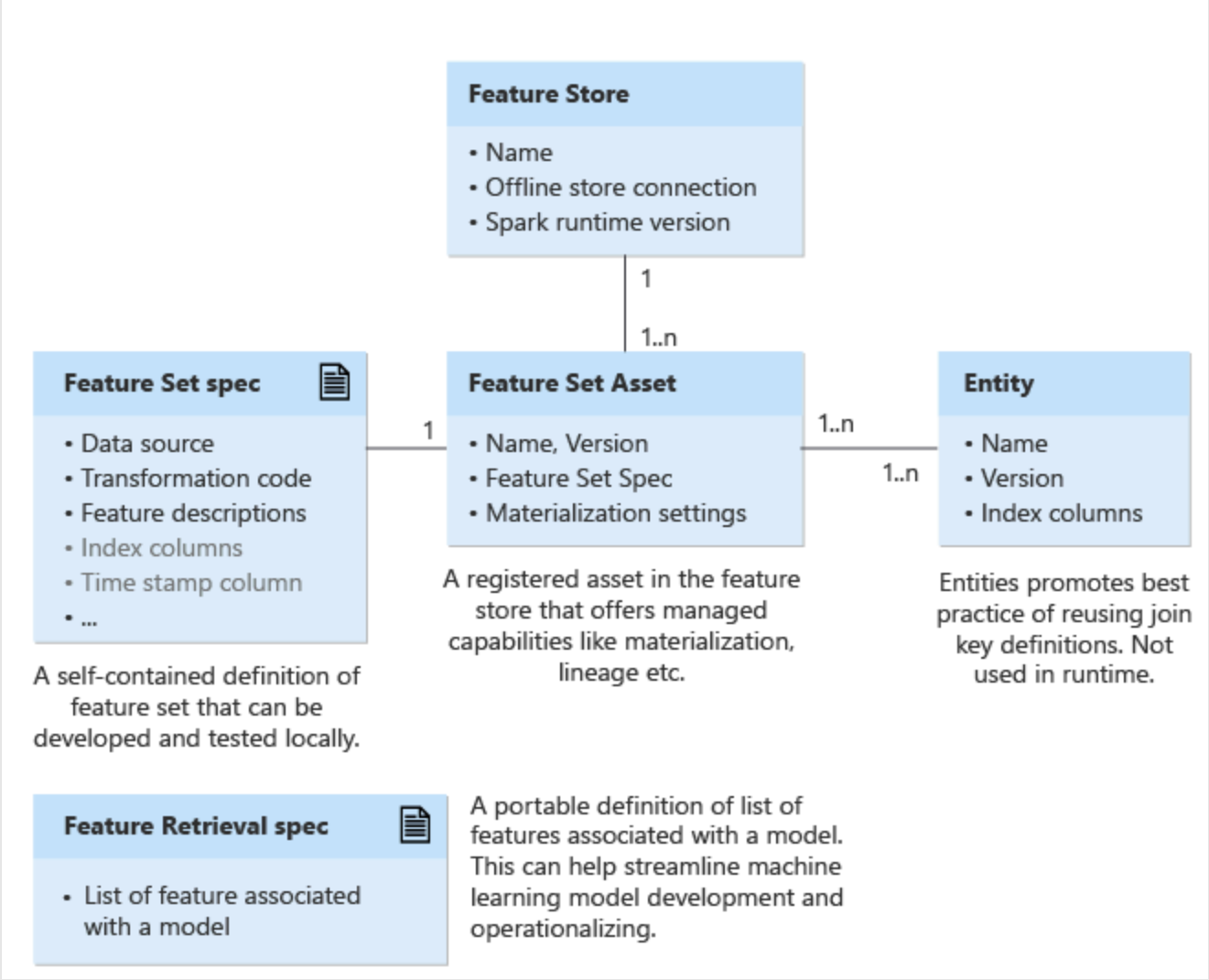
Understanding top-level entities in managed feature store

This article describes the top level entities in the managed feature store.



For more information on the managed feature store, see What is managed feature store?.

Feature store

You can create and manage feature sets through a feature store. A feature set is a collection of features. You can optionally associate a materialization store (offline store connection) with a feature store, to precompute and persist the features regularly. This approach can make feature retrieval during training or inference faster and more reliable.

For more information about the configuration, see the CLI (v2) feature store YAML schema resource.

Entities

An entity encapsulates the index columns for logical entities in an enterprise. Examples of entities include account entity, customer entity, and so on. Entities help enforce, as a best practice, the use of the same index column definitions across the feature sets that use the same logical entities.

Typically, you create entities once, then reuse them across feature sets. Entities are versioned.

For more information about the configuration, see the CLI (v2) feature entity YAML schema resource.

Feature set specification and asset

A feature set is a collection of features generated by applying a transformation on source system data. Feature sets encapsulate a source, the transformation function, and the materialization settings. Currently, the feature store supports PySpark feature transformation code.

First, create a feature set specification. A feature set specification is a self-contained definition of a feature set that you can develop and test locally.

A feature set specification typically consists of these parameters:

- `source`: The sources that this feature maps to
- `transformation` (optional): The transformation logic, applied to the source data, to create features. In this case, Spark is the supported compute.
- Names of the columns that represent the `index_columns` and the `timestamp_column`: These names are required when users try to join feature data with observation data (more about this later)
- `materialization_settings` (optional): Required if you want to cache the feature values in a materialization store for efficient retrieval.

After you develop and test the feature set spec in your local or development environment, you can register the spec as a feature set asset with the feature store. The feature set asset provides managed capabilities, such as versioning and materialization.

For more information about the feature set YAML specification, see the CLI (v2) feature set specification YAML schema resource.

Feature retrieval specification

A feature retrieval specification is a portable definition of a feature list that is associated with a model. It can help streamline machine learning model development and operationalization. A feature retrieval specification is typically an input to the training pipeline. It helps generate the training data. You can package it with the model. Additionally, the inference step uses it to look up the features. It integrates all phases of the machine learning lifecycle. Changes to your training and inference pipeline can be minimized as you experiment and deploy.

Use of a feature retrieval specification and the built-in feature retrieval component are optional. You can directly use the `get_offline_features()` API if you want.

For more information about the feature retrieval YAML specification, see the CLI (v2) feature retrieval specification YAML schema resource.

Next steps

- What is managed feature store?
- Manage access control for managed feature store

Last updated on 11/14/2025